ĐỀ CƯƠNG ÔN TẬP CHƯƠNG TRÌNH DỊCH

….Jan….

I. PHẦN LÝ THUYẾT

Câu 1: Trình bày định nghĩa chương trình dịch và phân loại chương trình dịch

- **Định nghĩa:** chương trình dịch là một chương trình dùng để chuyển một chương trình từ một ngôn ngữ (gọi là ngôn ngữ nguồn) thành một chương trình tương đương trong một ngôn ngữ khác (gọi là ngôn ngữ đích)
- **Phân loại chương trình dịch:** có thể phân thành nhiều loại tùy theo tiêu chí đưa ra để phân loại :

- Theo số lần duyệt :duyệt đôi, duyệt nhiều lần
- Theo mục đích: tải và chạy , gỡ rối , tối ưu , chuyển đổi ngôn ngữ,, chuyển đổi định dạng…
- Theo độ phức tạp của chương trình nguồn và chương trình dích:

+ Assembler (chương trình hợp dịch) dịch từ ngôn ngữ Assembly ra ngôn ngữ máy. Assembly là một ngôn ngữ cấp thấp, rất gần với ngôn ngữ máy.

+ Preprocessor (tiền xử lí) dịch từ ngôn ngữ cấp cao ra ngôn ngữ cấp cao khác. Thực chất chỉ dịch một số cấu trúc mới sang cấu trúc cũ.

+ Compiler (biên dịch): dịch từ ngôn ngữ cấp cao sang ngôn ngữ cấp thấp.

❖ Theo phương pháp dịch – chạy:

+ Thông dịch, còn gọi là diễn giải (interpreter) hành động do câu lệnh của ngôn ngữ quy định được thực hiện trực tiếp. Thông thường với mỗi hành động đều có tương ứng một chương trình con để thực hiện nó.

VD: Bộ lệnh của DOS, Fox Pro có thể chạy theo chế độ thông dịch.

+ Biên dịch: chương trình nguồn được dịch toàn bộ thành chương trình đích rồi mới chạy.

❖ Theo lớp văn phạm:

+ LL(1)

+ LR(1)

Tuy có nhiều cách phân loại, các chương trình dịch là giống nhau về nguyên lý. Chúng ta có thể tạo ra nhiều loại chương trình dịch cho các ngôn ngữ nguồn khác nhau , chương trình đích chạy trên các loại máy tính khác nhau mà vẫn sử dụng cùng một kỹ thuật cơ bản.

Câu 2 Trình bày giai đoạn phân tích , giai đoạn tổng hợp của chương trình dịch

❖ **Giai đoạn phân tích:**

1. **Phân tích từ vựng (lexical analyzer)** đọc luồng ký tự tạo thành chương trình nguồn từ trái sang phải, nhóm thành các ký hiệu mới ta gọi là từ tố như là tên ,số hay các phép toán.

2. **Phân tích cú pháp (syntax analyzer**) phân tích cấu trúc ngữ pháp của chương trình. Các từ tố sẽ được nhóm lại theo các cấu trúc phân cấp . Đôi khi ta gọi đây là phần phân tích phân cấp.
3. **Phân tích ngữ nghĩa (sematic analyzer)** phân tích tất cả các đặc tính khác của chương trình mà không thuộc đặc tính cú pháp. Nó kiểm tra chương trình nguồn để tìm những lỗi ngữ nghĩa và sự hợp kiểu (vd như không được gán bàn ghi cho số nguyên tố)

>>> Hai giai đoạn phân tích cú pháp và phân tích ngữ nghĩa có thể hoạt động như hai chức năng tách rời hoặc kết hợp làm một

- **Giai đoạn tổng hợp** : chương trình đích được sinh ra từ các ngôn ngữ trung gian theo các bước sau:
  1. Sinh mã trung gian (intermediate code generator) sinh chương trình trong ngôn ngữ trung gian nhằm mục đích: dễ sinh và tối ưu hơn mã máy và dễ chuyển đổi về mã máy hơn.
  2. Tối ưu mã (code optimize): sửa đổi chương trình trong ngôn ngữ trung gian nhằm cải tiến chương trình đích về hiệu năng.
  3. Sinh mã (code generator): tạo ra chương trình đích từ chương trình đích từ chương trình trong ngôn ngữ trung gian đã tối ưu.

<u>Câu 4 trình bày thuật toán tính FIRST(X) và FIRST(α)</u>

- **<u>Tính FiRST(X)</u>** ta sử dụng các quy tắc sau cho đến khi không còn kí hiệu kết thúc hoặc ký hiệu ε còn có thể thêm được vào một tập FIRST nào đó:
  1. Nếu X là ký hiệu kết thúc thì <u>FIRST(X)</u> = {X}
  2. Nếu X → ε là một sản xuất thì thêm ε vào FIRST(X)
  3. Nếu X → $Y_1Y_2...Y_k$ là một sản xuất và nếu với một i nào đó thì ε có trong mọi FIRST(Y_1) , FIRST(Y_2),…. FIRST(Y_{i-1}) (nghĩa là $Y_1Y_{2...}Y_{i-1}$ → ε) thì ta thêm mọi ký hiệu kết thúc có trong First(Y_i) vào First(X). Nếu i=k(tức là ε có trong mọi FIRST(Y_i) với i=1,2,…, k.) thì thêm ε vào FIRST(X)

- **<u>Tính FIRST(α)</u>**

Thêm vào FIRST($X_1X_2...X_n$) tất cả các ký hiệu không phải ε của FIRST(X_1). Ta cũng thêm các ký hiệu không phải ε của FIRST(X_2) nếu ε thuộc FIRST(X_1) , các ký hiệu không phải ε của FIRST(X_3) nếu ε thuộc FIRST(X_1), FIRST(X_2), cứ như vậy … cuối cùng thêm ε vào FIRST($X_1X_2...X_n$) nếu với mọi i mà mọi FIRST(X_i) có chứa ε hoặc n=0.

<u>Câu 5: trình bày thuật toán tính Follow(A) và lập bảng phân tích</u>

- **<u>Tính FOLLOW (A)</u>** ta dùng các quy tắc sau cho đến khi không thể thêm gì vào tập FOLLOW
  1. Đặt $ vào FOLLOW(A), với A là ký hiệu bắt đầu (đỉnh cây) , $ là ký hiệu đánh dấu kết thúc xâu vào(chú ý là A không nhất thiết phải trùng với S do ta đang tính FOLLOW cho cây con)

2. Nếu có một sản xuất dạng B→**α**Aß (với ß ≠ ε) thì mọi phần tử thuộc FIRST(ß) trừ ε đều được cho vào FOLLOW(A)
3. Nếu có một sản xuất dạng B→αA (hoặc một sản xuất B→**α**Aß với FIRST(ß) chứa ε , hay ß→ ε) thì mọi phần tử của FOLLOW(B) cũng cho vào FOLLOW(A)

❖ **Lập bảng phân tích**

- Thuật toán dùng để phân tích tất định cho văn phạm G

Cho A→**α** là một sản xuất với a thuộc FIRST(**α**). Mỗi khi bộ phân tích gặp A ở trên đỉnh của ngăn xếp và a là ký hiệu vào hiện tại thì bộ phân tích sẽ mở rộng A bằng **α.** Chỉ có một sự rắc rối khi **α**= ε hay **α**→ ε. Trong trường hợp này, chúng ta cũng có thể mở rộng A bằng a nếu như ký hiệu vào hiện tại thuộc FOLLOW(A) hoặc nếu con trỏ đầu vào chỉ đến $ và $ thuộc FOLLOW(A)

>>>> Thuật toán xây dựng bảng phân tích tất định LL(1)

Vào: văn phạm G

Ra: bảng phân tích M

Thuật toán:

1. Đối với mỗi sản xuất dạng A →**α**, thực hiện bước 2 và bước 3
2. Đối với mỗi ký hiệu kết thúc a thuộc FIRST(**α)** , thêm A→**α** vào M[A,a]
3. Nếu ε thuộc FIRST(**α**) , thêm A →**α** vào M[A,b] đối với mỗi b thuộc FOLLOW(A). Nếu ε thuộc FIRST(**α)** và $ là thuộc FOLLOW(A) , thêm A →**α** vào M[A, $]
4. Đặt tất cả các vị trí chưa được định nghĩa còn lại của bảng là lỗi

Câu 6: Trình bày điều kiện để một văn phạm là LL(1), giải thích ý nghĩa của từng điều kiện

>>> Điều kiện để một văn phạm là LL(1)

Nếu **A→ α|** ß là hai sản xuất phân biệt của G thì các điều kiện sau phải thỏa mãn : (Mục đích : mỗi ô có một giá trị)

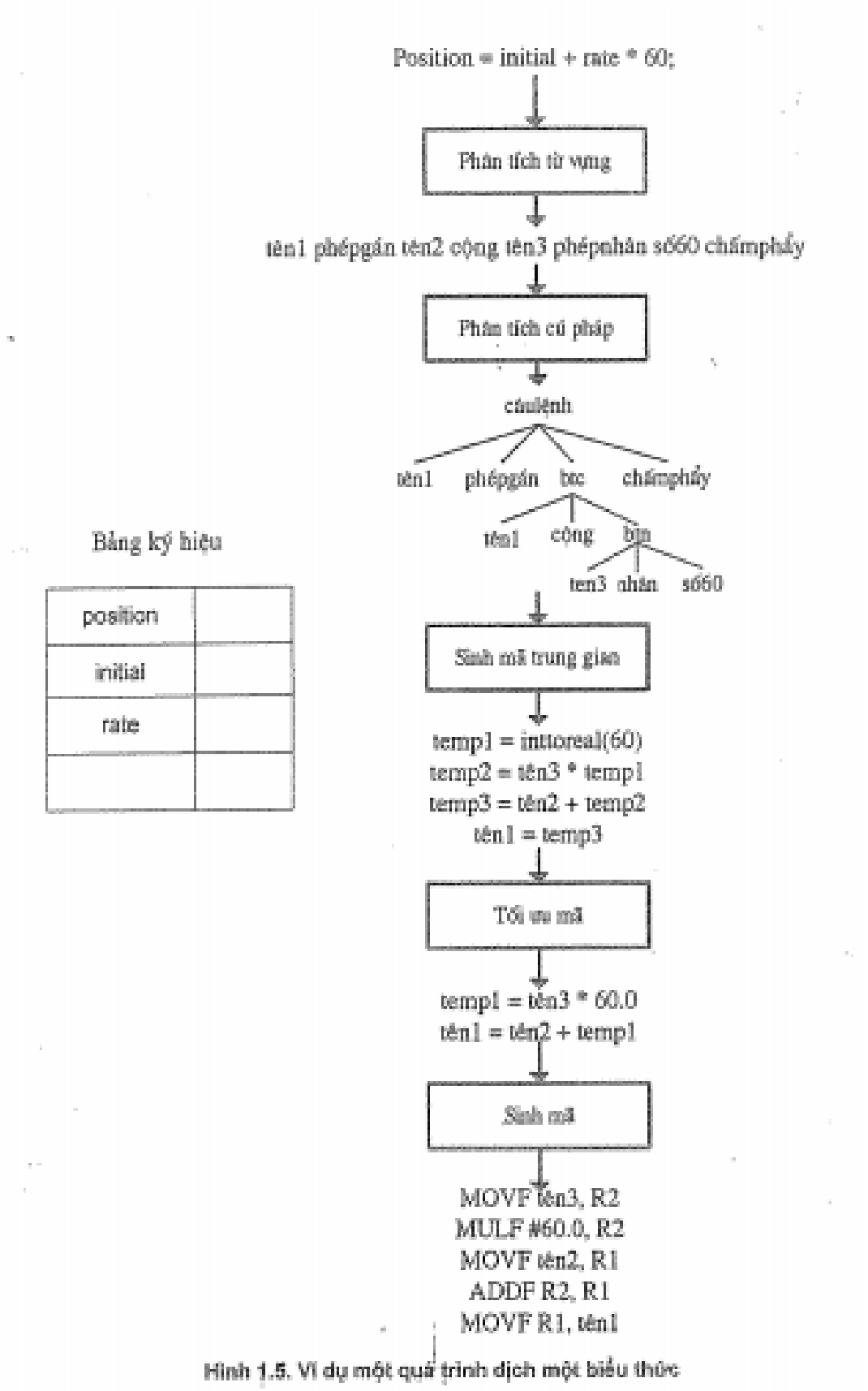
1. Không có một ký hiệu kết thúc a nào mà cả **α** và ß có thể suy dẫn các xâu bát đầu bằng a
   - ⇨ First phải khác nhau => cho 1 giá trị
2. Nhiều nhất là chỉ một trong **α** hoặc ß có thể suy dẫn ra xâu rỗng.
3. Nếu ß→ ε thì **α** không suy dẫn được một xâu nào bắt đầu bằng một ký hiệu kết thúc b thuộc FOLLOW(A)
   - ⇨ First(A) và Follow(A) không được giao nhau.

Một cách đơn giản khác để kiểm tra xem một văn phạm có là LL(1) hay không là lập bảng phân tích và dựa vào định nghĩa sau để kiểm tra.

Định nghĩa văn phạm LL(1) là csc văn phạm xây dựng được bảng phân tích theo thuật toán xây dựng bảng phân tích tất định LL có các ô chỉ được định nghĩa nhiều

nhất là một lần. Như vậy văn phạm LL(10 không bị nhập nhằng và không có đệ quy trái.

Câu 3: vẽ hình ví dụ một quá trình dịch một biểu thức



Hình 1.5. Ví dụ một quá trình dịch một biểu thức

4. Trình bày phương pháp phân tích Top Down (trang 60).

5. Trình bày phương pháp phân tích Bottom up (trang 62-63)

9. Trình bày mục đích, nhiệm vụ của bảng ký hiệu trong chương trình dịch, các khả năng làm việc với bảng ký hiệu (trang 140,141) (không cần vẽ hình).

10. Trình bày cách tổ chức bảng ký hiệu bằng cây định danh và danh sách phạm vi (trang 245, 246) (không cần vẽ hình).

II. PHẦN BÀI TẬP VÀ VIẾT CODE

1. Cho tập sản xuất của một văn phạm. Hãy:

a) Tính FIRST và FOLLOW cho các ký hiệu không kết thúc

b) Lập bảng phân tích tất định LL(1) cho văn phạm trên. Văn phạm trên có phải là văn phạm LL(1) hay không?

c) Phân tích LL với xâu vào nào đó theo dạng:

Ngăn xếp	Đầu vào	Đầu ra
. . .	. . .	. . .

2. Sửa lại phần khai báo từ tố, khai báo từ khóa và viết lại hàm next_token() của chương trình phân tích từ vựng (theo phương pháp diễn giải đồ thị chuyển) chỉ cho một phần nào đó của ngôn ngữ SLANG.

Ví dụ: Sửa lại phần khai báo từ tố, khai báo từ khóa và và viết lại hàm next_token() của chương trình phân tích từ vựng (theo phương pháp diễn giải đồ thị chuyển) chỉ cho câu lệnh WHILE (không quan tâm đến các câu lệnh trong thân của WHILE). Phần <quan hệ> của WHILE chỉ có các phép so sánh: > , >=, = . Phần <biểu thức> trong <quan hệ> chỉ có các phép tính cộng, trừ. Phần <toán hạng> trong <biểu thức> chỉ có tên và số. (*Chú ý:* Chương trình luôn bắt đầu bằng từ khóa BEGIN và kết thúc là END.)

Bài làm

SLC.H

typedef enum {

Begin_token, end_token, ident_token, num_token, while_token, do_token, od_token, greater_than_token, greater_or_equal_token, plus_token,minus_token, become_token,separator_token, period_token.

} token;

SLC.C

Struct keyword keywtb[]=

{

{begin_token, “BEGIN”}, {end_token,”END”}, {while_token,”WHILE”}, {do_token,”DO”}, {od_token,”OD”},

{begin_token, “BEGIN”}, {end_token,”END”}, {if_token,"IF”}, {then_token,”THEN”}, {else_token,"ELSE"},{fi_token,”FI”}

}

void next_token(void)

```
 {
   retract=false;
   //Loại bỏ các ký tự trắng, cách, tab, xuống dòng, chú thích
   while ((ch<=' '||ch=='{')&& ch!=EOF)
     {
       if (ch=='{') comment();
       next_character();
     }
   if (ch==EOF) return;
   switch(ch)
     {


       case '.':
           look_ahead=period_token;
           break;
```

```
    case ';':
        look_ahead=separator_token;
        break;
   case '>':
        next_character();
        if(ch=='=') look_ahead=greater_or_equal_than_token;
          else
            {
           look_ahead=greater_than_token;
             retract=true;
            }
        break;
    default:
        if (isalpha(ch))
          identifier();
        else
          if(isdigit(ch))
            number();
   }
  if(!retract) next_character();
 }
```

3. **Trong chương trình phân tích từ vựng theo phương pháp diễn giải đồ thị chuyển cho ngôn ngữ SLANG:**
   a) **Trình bày công dụng của hàm identifier() và biến ident_lexeme. Viết nội dung hàm identifier().**
   b) **Trình bày công dụng của hàm number() và biến num_value. Viết nội dung hàm number().**
   c) **Trình bày công dụng của hàm look_up()**

Bài làm

a)

- Công dụng của hàm identifier(): phân tích tên và kiểm tra xem có phải là từ khóa không. Trả về từ tố
- Công dụng của biến ident lexeme : lưu trữ từ khóa hoặc tên
- Nội dung hàm:

void identifier(void)

```
{
  int index=0;
  do
   {
    ident_lexeme[index++]=ch;
    next_character();
   }
  while (isalpha(ch)||isdigit(ch));
  ident_lexeme[index]=0; //Thêm ký tự kết thúc xâu
  look_ahead=look_up(ident_lexeme);
  if (look_ahead==err_token)
    look_ahead=ident_token;
  retract=true;
}
```

b)

- công dụng hàm number(): đoán nhận số, tính giá trị số và lưu vào biến num_value
- công dụng của biến num_value: lưu trữ số được đoán nhận
- nội dung hàm:

void number(void)

```
{
  num_value=0;
  do
   {
    num_value=num_value*10+ch-'0';
    next_character();
   }
  while (isdigit(ch));
  look_ahead=num_token;
  retract=true;
}
```

c) công dụng hàm look_up:trả lại từ tố của từ khóa có trong biến ident_lexeme

4. **Viết nội dung một số hàm của chương trình phân tích cú pháp theo phương pháp đệ quy trên xuống cho ngôn ngữ SLANG.**

Bài làm

```
void program_declaration(void)
 {
  printf("program_declaration\n");
  block();
  if (look_ahead=period_token) next_token();
 }
void block(void)
 {
  printf("block\n");
  if (look_ahead==begin_token) next_token();
  declaration_part();
  statement_part();
  if (look_ahead==end_token) next_token();
 }
void declaration_part(void)
 {
  printf("declaration_part\n");
  while (look_ahead==int_token || look_ahead==var_token)
   {
    if (look_ahead==int_token)
      constant_declaration();
    else if (look_ahead==var_token)
      variable_declaration();
   }
  while (look_ahead==procedure_token)
   procedure_declaration();
 }
void constant_declaration(void)
 {
```

```
  printf("constant_declaration\n");
  type_declarer();
  if (look_ahead==ident_token) next_token();
  if (look_ahead==equal_token) next_token();
  if (look_ahead==num_token) next_token();
  while (look_ahead==list_token)
    {
      if (look_ahead==list_token) next_token();
      if (look_ahead==ident_token) next_token();
      if (look_ahead==equal_token) next_token();
      if (look_ahead==num_token) next_token();
    }
  if (look_ahead==separator_token) next_token();
 }
void type_declarer(void)
 {
  printf("type_declarer\n");
  if (look_ahead==int_token) next_token();
 }
void variable_declaration(void)
 {
  printf("variable_declaration\n");
  if (look_ahead==var_token) next_token();
  type_declarer();
  if (look_ahead==ident_token) next_token();
  while (look_ahead==list_token)
    {
      if (look_ahead==list_token) next_token();
      if (look_ahead==ident_token) next_token();
    }
  if (look_ahead==separator_token) next_token();
 }
void procedure_declaration(void)
 {
  printf("procedure_declaration\n");
```

```
  if (look_ahead==procedure_token) next_token();
  if (look_ahead==ident_token) next_token();
    block();
  if (look_ahead==separator_token) next_token();
 }
void statement_part(void)
 {
  printf("statement_part\n");
  statement();
  while (look_ahead==separator_token)
    {
      if (look_ahead==separator_token) next_token();
      statement();
    }
 }
void statement(void)
 {
  printf("statement\n");
  if (look_ahead==ident_token) assignment_statement();
  else if (look_ahead==if_token) if_statement();
  else if (look_ahead==while_token) while_statement();
  else if (look_ahead==call_token) call_statement();
  else if (look_ahead==read_token) read_statement();
  else if (look_ahead==write_token) write_statement();
 }
void assignment_statement(void)
 {
  printf("assignment_statement\n");
  left_part();
  if (look_ahead==become_token) next_token();
  expression();
 }
void left_part(void)
 {
  printf("left_part\n");
```

```
  if (look_ahead==ident_token) next_token();
 }
void if_statement(void)
 {
  printf("if_statement\n");
  if (look_ahead==if_token) next_token();
  relation();
  if (look_ahead==then_token) next_token();
  statement_part();
  if (look_ahead==else_token)
   {
    if (look_ahead==else_token) next_token();
      statement_part();
   }
  if (look_ahead==fi_token) next_token();
 }
void while_statement(void)
 {
  printf("while_statement\n");
  if (look_ahead==while_token) next_token();
  relation();
  if (look_ahead==do_token) next_token();
  statement_part();
  if (look_ahead==od_token) next_token();
 }
void call_statement(void)
 {
  printf("call_statement\n");
  if (look_ahead==call_token) next_token();
  if (look_ahead==ident_token) next_token();
 }
void read_statement(void)
 {
  printf("read_statement\n");
  if (look_ahead==read_token) next_token();
```

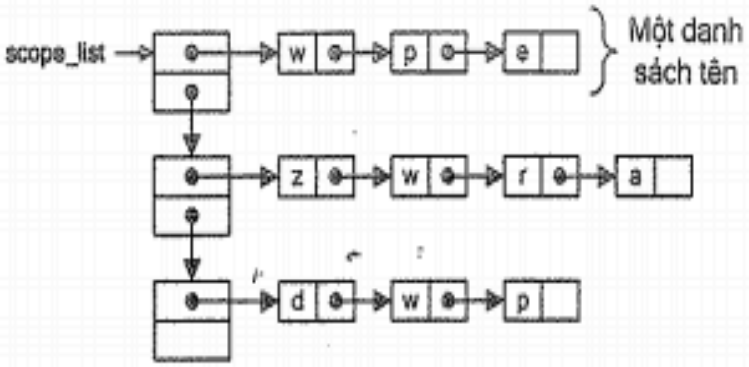
```
  if (look_ahead==open_token) next_token();
  if (look_ahead==ident_token) next_token();
  while (look_ahead==list_token)
   {
     if (look_ahead==list_token) next_token();
     if (look_ahead==ident_token) next_token();
   }
  if (look_ahead==close_token) next_token();
 }
void write_statement(void)
 {
  printf("write_statement\n");
  if (look_ahead==write_token) next_token();
  if (look_ahead==open_token) next_token();
  expression();
  while (look_ahead==list_token)
   {
     if (look_ahead==list_token) next_token();
     expression();
   }
  if (look_ahead==close_token) next_token();
 }
void expression(void)
 {
  printf("expression\n");
  term();
  while (look_ahead==minus_token|| look_ahead==plus_token)
   {
     add_operator();
     term();
   }
 }
void term(void)
 {
  printf("term\n");
```

```
  factor();
  while (look_ahead==modulo_token|| look_ahead==over_token||
        look_ahead==times_token)
    {
      multiply_operator();
      factor();
    }
 }
void factor(void)
 {
  printf("factor\n");
  while (look_ahead==absolute_token|| look_ahead==negate_token)
    unary_operator();
  operand();
 }
void operand(void)
 {
  printf("operand\n");
  if (look_ahead==ident_token) next_token();
  else if (look_ahead==num_token) next_token();
  else if (look_ahead==open_token)
    {
      if (look_ahead==open_token) next_token();
      expression();
      if (look_ahead==close_token);
    }
 }
void add_operator(void)
 {
  printf("add_operator\n");
  if (look_ahead==plus_token) next_token();
  else if (look_ahead==minus_token) next_token();
 }
void multiply_operator(void)
 {
```

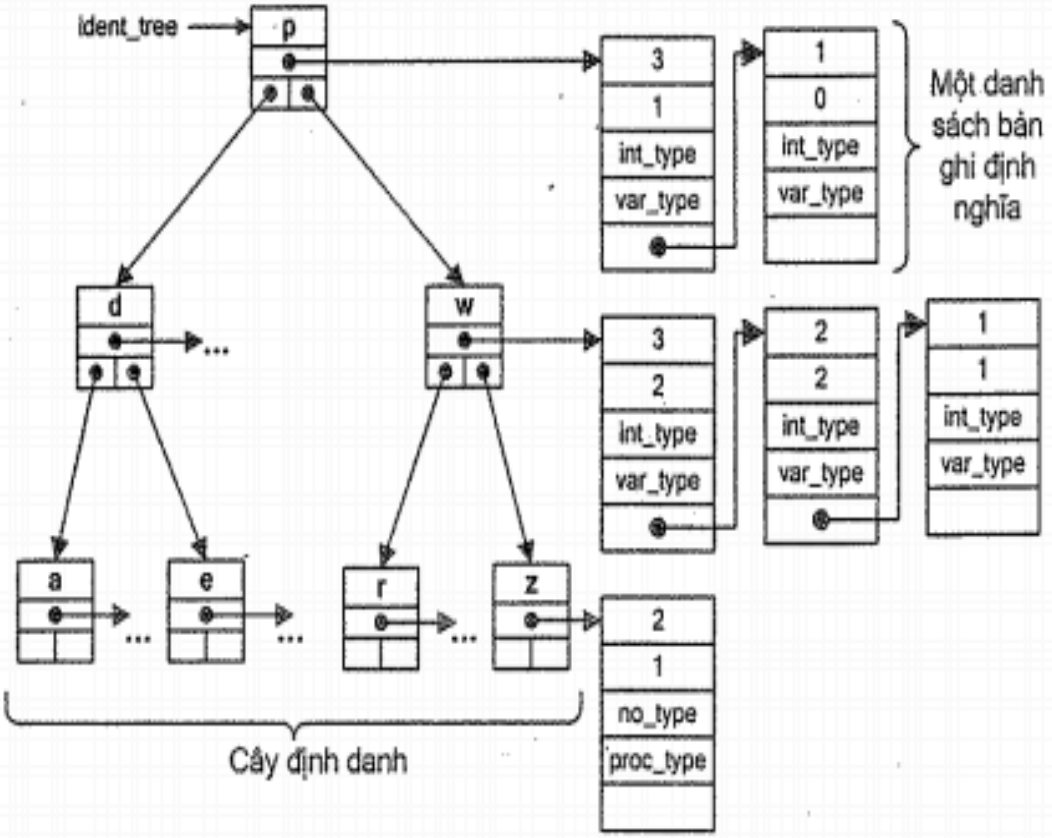
```
  printf("multiply_operator\n");
  if (look_ahead==times_token) next_token();
  else if (look_ahead==over_token) next_token();
  else if (look_ahead==modulo_token) next_token();
 }
void unary_operator(void)
 {
  printf("unary_operator\n");
  if (look_ahead==negate_token) next_token();
  else if (look_ahead==absolute_token) next_token();
 }
void relation(void)
 {
  printf("relation\n");
  expression();
  relation_operator();
  expression();
 }
void relation_operator(void)
 {
  printf("relation_operator\n");
  if (look_ahead==equal_token) next_token();
  else if (look_ahead==not_equal_token) next_token();
  else if (look_ahead==less_than_token) next_token();
  else if (look_ahead==less_or_equal_token) next_token();
  else if (look_ahead==greater_than_token) next_token();
  else if (look_ahead==greater_or_equal_token) next_token();
 }
```

5. **Cho trước một chương trình viết bằng ngôn ngữ SLANG. Hãy lập bảng ký hiệu theo cách dùng cây định danh và danh sách phạm vi khi phân tích đến dòng nào đó. (VD trang 249)**

Bài làm



Hình 5.2. Cây định danh và các danh sách bản ghi định nghĩa.



Hình 5.3. Danh sách phạm vi và các danh sách bản ghi tên.

6. **Trình bày hàm xử lý khi ra khỏi chương trình con exit_scope() (trang 257-258)**

Bài làm

```
void exit_scope(void)
{
        scope_ptr       scp;
        name_ptr        np1, np2;
        ident_ptr       ip;
        def_ptr  dp;
        int             k;

        /* Phần này tìm và xóa tất cả các bản ghi trong danh sách tên
                ứng với phạm vi trên cùng */
        for (np1 = scope_list->name_list; np1 != NULL; np1 = np2) {
                np2 = np1->next;
        /* Phần này tìm và xóa bản ghi đầu tiên trong danh sách
                bản ghi định nghĩa ứng với tên của bản ghi tên - để
                làm điều này đòi hỏi phải duyệt cây định danh */
                for (ip=ident_tree; ip!= NULL;) {
                        k = stricmp(ip->ident, np1->ident);
                        if (k == 0) {
                                dp = ip->def_list;
                                ip->def_list = dp->next;
                                free(dp);
                                break;
```

```
                        }
                        if (k > 0) ip = ip->left; else ip = ip->right;
                }

                free(np1);
        }
        /* Xóa bản ghi trên cùng của danh sách phạm vi */
        scp = scope_list;
        scope_list = scope_list->prev_scope;
        free(scp);
        scope_level--;
}

int current_scope(void)
{
        return scope_level;
}

def_ptr create_definition (int sn, int dpl_or_value_or_index, e_type type, e_kind kind)
{
        def_ptr   dp;

        dp = (def_ptr) malloc(sizeof(struct def_rec));
        dp->sn = sn;
        dp->dpl_or_value_or_index = dpl_or_value_or_index;
        dp->type = type;
        dp->kind = kind;
        dp->next = NULL;

        return dp;
}
```

7. **Trình bày hàm chèn thêm bản ghi mới vào bảng ký hiệu insert_definition() (trang 258-259)**

Bài làm

```
void insert_definition (string ident, def_ptr def)
{
        ident_ptr ip1, ip2;
        name_ptr np;
        int       k;

        /* Tìm bản ghi ứng với tên trong cây định danh */
        ip2 = ident_tree;
        while (ip2 != NULL) {
                ip1 = ip2;
                k = stricmp(ip1->ident, ident);
                if (k == 0) break;
                if (k > 0) ip2 = ip1->left; else ip2 = ip1->right;
        }

        /* Nếu tìm thấy thì thêm bản ghi định nghĩa trỏ bởi def vào
        đầu danh sách định nghĩa */
        if (k==0 && ident_tree== NULL) {
```

```
        def->next = ip1->def_list;
        ip1->def_list = def;
}
else {
        /* Nếu không thấy thì tạo nút mới cho cây định danh */
        ip2 = (ident_ptr) malloc(sizeof(struct ident_rec));
        strcpy(ip2->ident, ident);
        ip2->def_list = def;
        ip2->left = ip2->right = NULL;

        if (ident_tree == NULL) ident_tree = ip2; /* Trường hợp
                                                     nút đầu tiên của cây  */
        else
        if (k > 0) ip1->left = ip2; else ip1->right = ip2;
}

/* Kiểm tra xem tên đã cho có sẵn trong danh sách tên của
  mức phạm vi hiện tại hay không (kiểm tra tính duy nhất) */
for (np = scope_list->name_list; np != NULL; np = np->next)
        if (stricmp(ident, np->ident)==0) {
                report_error("Tên được định nghĩa hai lần");
                return;
        }

/* Nếu chưa thì thêm vào đầu danh sách tên */
np = (name_ptr) malloc(sizeof(struct name_rec));
strcpy(np->ident, ident);
np->next = scope_list->name_list;
scope_list->name_list = np;
```

8. **Cho trước một chương trình viết bằng ngôn ngữ SLANG. Giả sử đang phân tích đến vị trí nào đó. Hãy:**
   - **Trình bày các thủ tục cú pháp và các tập đồng bộ tương ứng (trang 288).**
   - **Trình bày chiến lược khôi phục lỗi khi thừa hoặc thiếu từ tố nào đó (trang 289-290)**

Bài làm

- Các thủ tục cú pháp và các tập đồng bộ tương ứng

Tập đồng bộ tức thời

relation: DIRSET(*relation*)

Tập đồng bộ cục bộ

if_statement: { then_token } ∪ DIRSET(*statement_part*) ∪ DIRSET([else_token *statement_part*]) ∪ { fi_token }

Tập đồng bộ toàn cục

statement: ε

statement_part: DIRSET((*separator_token statement*)*)

while_statement: {od_token}

statement: ε

statement_part: DIRSET((separator_token statement)*)

block: {end_token}

program_declaration: {period_token}

Thực hiện các phép tính trên với SLANG ta có:

Tập đồng bộ tức thời: {ident_token, num_token, open_token, absolute_token, negate_token}

Tập đồng bộ cục bộ: {then_token, else_token, fi_token, ident_token, if_token, while_token, call_token, read_token, write_token }

Tập đồng bộ toàn cục: {separator_token, od_token, end_token, period_token}

- Các chiến lược khôi phục lỗi khi thừa hoặc thiếu từ tố nào đó

Chiến lược khôi phục lỗi như sau: nếu ký hiệu nhìn trước không được mong đợi, nhưng lại có trong các tập đồng bộ hiện thời, thì bộ phân tích coi là một từ tố bị mất, nó chèn thêm từ tố này vào và tiếp tục phân tích. Nếu như từ tố này không có trong các tập đồng bộ, thì bộ phân tích coi ký hiệu nhìn trước này là thừa và xoá nó đi, sau đó lại tiếp tục phân tích. Ta sẽ minh hoạ bằng hai ví dụ dưới đây.

Ví dụ 7.2: Giả sử từ tố then _token bị mất trong ví dụ trên, nghĩa là câu lệnh phân tích chỉ là:

IF a < b a := b+2 ELSE b := a-2 FI; c := a

Ký hiệu nhìn trước (nghĩa là ident _token ứng với vế trái của câu lệnh gán thứ nhất) có trong các tập đồng bộ cục bộ. Do vậy, bộ phân tích coi là từ tố then _token bị mất, nó sẽ chèn thêm từ tố này vào và in ra một thông báo: "Lỗi: chèn thêm then _token" và làm việc tiếp tục.

Ví dụ 7.3: Trong ví dụ này, giả sử từ tố then _token được thay bằng do _token, nghĩa là câu lệnh trở thành:

IF a < b DO a := b+2 ELSE b := a-2 FI; c := a

Ký hiệu nhìn trước lúc này là từ tố do _token sẽ không có trong các tập đồng bộ. Do đó, bộ phân tích coi từ tố do _token là dư thừa nên sẽ xoá nó đi, và in ra thông báo lỗi "Lỗi: xoá do _token". Tiếp theo, bộ phân tích kết luận là từ tố then _token bị thiếu và xử lý giống như ví dụ trên.

SLC.H // Thư viện full phân tích từ vựng

```
typedef enum {
  ident_token, num_token, begin_token, end_token, int_token, var_token,
  procedure_token, call_token, read_token, write_token, if_token,
then_token, else_token, fi_token, while_token, do_token, od_token,
negate_token,absolute_token, open_token, close_token, list_token,
period_token,separator_token, become_token, plus_token, minus_token,
times_token, over_token,modulo_token, equal_token, not_equal_token,
less_than_token, less_or_equal_token, greater_than_token,
greater_or_than_token, err_token
 } token;
```

#TM598